

PSG COLLEGE OF TECHNOLOGY, COIMBATORE -641004

DEPARTMENT OF COMPUTER APPLICATIONS

I SEMESTER MCA

PROBLEM SHEET ON STRINGS

**Tino Britty J
D25MX316**

Question 1

Given a string *s* consisting of words and spaces, return the length of the last word in the string. A word is a maximal substring consisting of non-space characters only.

Example 1:

Input: *s* = "Hello World"

Output: 5

Explanation: The last word is "World" with length 5.

Example 2:

Input: *s* = " fly me to the moon "

Output: 4

Explanation: The last word is "moon" with length 4.

Example 3:

Input: *s* = "luffy is still joyboy"

Output: 6

Explanation: The last word is "joyboy" with length 6.

Constraints:

$1 \leq s.length \leq 104$

s consists of only English letters and spaces ' '.

There will be at least one word in *s*.

```
public class LastWordLength {  
  
    public int lengthOfLastWord(String s) {  
        s = s.trim();  
        int lastSpace = s.lastIndexOf(' ');  
        return s.length() - lastSpace - 1;  
    }  
  
    public static void main(String[] args) {  
        LastWordLength solution = new LastWordLength();  
    }  
}
```

```

        System.out.println(solution.lengthOfLastWord("Hello World")); //
Output: 5

        System.out.println(solution.lengthOfLastWord(" fly me to the
moon ")); // Output: 4

        System.out.println(solution.lengthOfLastWord("luffy is still
joyboy")); // Output: 6
    }
}

```

Question 2

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

Input: *s* = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: *s* = "race a car"

Output: false

Explanation: "raceacar" is not a palindrome.

Example 3:

Input: *s* = ""

Output: true

Explanation: *s* is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

$1 \leq s.length \leq 2 * 10^5$

s consists only of printable ASCII characters.

```

public class ValidPalindrome {

    public boolean isPalindrome(String s) {

        StringBuilder cleaned = new StringBuilder();

        for (char c : s.toCharArray()) {

```

```

        if (Character.isLetterOrDigit(c)) {
            cleaned.append(Character.toLowerCase(c));
        }
    }

    String cleanedStr = cleaned.toString();

    return cleanedStr.equals(cleaned.reverse().toString());
}

public static void main(String[] args) {
    ValidPalindrome solution = new ValidPalindrome();

    System.out.println(solution.isPalindrome("A man, a plan, a
canal: Panama")); // Output: true

    System.out.println(solution.isPalindrome("race a car")); //
Output: false

    System.out.println(solution.isPalindrome(" ")); // Output: true
}
}

```

Question 3

Given an input string *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by at least one space. Return a string of the words in reverse order concatenated by a single space.

Note that *s* may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: *s* = "the sky is blue"

Output: "blue is sky the"

Example 2:

Input: *s* = " hello world "

Output: "world hello"

Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: *s* = "a good example"

Output: "example good a"

Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Constraints:

1 <= s.length <= 104

s contains English letters (upper-case and lower-case), digits, and spaces ' '. There is at least one word in s.

```
public class ReverseWords {  
    public String reverseWords(String s) {  
        String[] words = s.trim().split("\\s+");  
        StringBuilder reversed = new StringBuilder();  
        for (int i = words.length - 1; i >= 0; i--) {  
            reversed.append(words[i]);  
            if (i > 0) reversed.append(" ");  
        }  
        return reversed.toString();  
    }  
  
    public static void main(String[] args) {  
        ReverseWords solution = new ReverseWords();  
        System.out.println(solution.reverseWords("the sky is blue")); //  
Output: "blue is sky the"  
        System.out.println(solution.reverseWords(" hello world ")); //  
Output: "world hello"  
        System.out.println(solution.reverseWords("a good example")); //  
Output: "example good a"  
    }  
}
```

Question 4

Given two strings s and t, determine if they are isomorphic.

Two strings s and t are isomorphic if the characters in s can be replaced to get t.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: s = "egg", t = "add"

Output: true

Explanation: The strings s and t can be made identical by: Mapping 'e' to 'a'. Mapping 'g' to 'd'.

Example 2:

Input: s = "foo", t = "bar"

Output: false

Explanation: The strings s and t cannot be made identical as 'o' needs to be mapped to both 'a' and 'r'.

Example 3:

Input: s = "paper", t = "title"

Output: true

Constraints:

$1 \leq s.length \leq 5 * 10^4$

$t.length == s.length$

s and t consist of any valid ascii character.

```
import java.util.HashMap;
import java.util.Map;

public class IsomorphicStrings {
    public boolean isIsomorphic(String s, String t) {
        if (s.length() != t.length()) return false;
        Map<Character, Character> map = new HashMap<>();
        Map<Character, Character> reverseMap = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            char c1 = s.charAt(i);
            char c2 = t.charAt(i);
            if (map.containsKey(c1)) {
                if (map.get(c1) != c2) return false;
            } else {
                map.put(c1, c2);
                reverseMap.put(c2, c1);
            }
        }
    }
}
```

```

        if (reverseMap.containsKey(c2)) {
            if (reverseMap.get(c2) != c1) return false;
        } else {
            reverseMap.put(c2, c1);
        }
    }

    return true;
}

public static void main(String[] args) {
    IsomorphicStrings solution = new IsomorphicStrings();

    System.out.println(solution.isIsomorphic("egg", "add")); //
Output: true

    System.out.println(solution.isIsomorphic("foo", "bar")); //
Output: false

    System.out.println(solution.isIsomorphic("paper", "title")); //
Output: true

}
}

```

Question 5

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Note: An anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

1 ≤ *s*.length, *t*.length ≤ 5 * 10⁴

s and *t* consist of lowercase English letters.

```

import java.util.Arrays;

public class ValidAnagram {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        char[] sChars = s.toCharArray();
        char[] tChars = t.toCharArray();
        Arrays.sort(sChars);
        Arrays.sort(tChars);
        return Arrays.equals(sChars, tChars);
    }

    public static void main(String[] args) {
        ValidAnagram solution = new ValidAnagram();
        System.out.println(solution.isAnagram("anagram", "nagaram")); //
Output: true
        System.out.println(solution.isAnagram("rat", "car")); // Output:
false
    }
}

```

Question 6

Given a pattern and a string s, find if s follows the same pattern. Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s. Specifically: Each letter in pattern maps to exactly one unique word in s. Each unique word in s maps to exactly one letter in pattern. No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation: The bijection can be established as: 'a' maps to "dog". 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

1 <= pattern.length <= 300

pattern contains only lower-case English letters.

1 <= s.length <= 3000

s contains only lowercase English letters and spaces ' '.

s does not contain any leading or trailing spaces.

All the words in s are separated by a single space.

```
import java.util.HashMap;
import java.util.Map;

public class WordPattern {
    public boolean wordPattern(String pattern, String s) {
        String[] words = s.split(" ");
        if (words.length != pattern.length()) return false;
        Map<Character, String> map = new HashMap<>();
        Map<String, Character> reverseMap = new HashMap<>();
        for (int i = 0; i < pattern.length(); i++) {
            char c = pattern.charAt(i);
            String word = words[i];
            if (map.containsKey(c)) {
                if (!map.get(c).equals(word)) return false;
            } else {
                map.put(c, word);
            }
            if (reverseMap.containsKey(word)) {
                if (reverseMap.get(word) != c) return false;
            } else {
                reverseMap.put(word, c);
            }
        }
    }
}
```



```

        return true;
    }

    public static void main(String[] args) {
        WordPattern solution = new WordPattern();

        System.out.println(solution.wordPattern("abba", "dog cat cat
dog")); // Output: true

        System.out.println(solution.wordPattern("abba", "dog cat cat
fish")); // Output: false

        System.out.println(solution.wordPattern("aaaa", "dog cat cat
dog")); // Output: false
    }
}

```

Question 7

Given a string array words, return the maximum value of $\text{length}(\text{word}[i]) * \text{length}(\text{word}[j])$ where the two words do not share common letters. If no such two words exist, return 0.

Example 1:

Input: words = ["abcw","baz","foo","bar","xtfn","abcdef"]

Output: 16

Explanation: The two words can be "abcw", "xtfn".

Example 2:

Input: words = ["a","ab","abc","d","cd","bcd","abcd"]

Output: 4

Explanation: The two words can be "ab", "cd".

Example 3:

Input: words = ["a","aa","aaa","aaaa"]

Output: 0

Explanation: No such pair of words.

Constraints:

$2 \leq \text{words.length} \leq 1000$

$1 \leq \text{words}[i].\text{length} \leq 1000$

words[i] consists only of lowercase English letters.

```
import java.util.HashSet;
```

```

import java.util.Set;

public class MaxProductWordLengths {

    public int maxProduct(String[] words) {

        int n = words.length;

        int[] masks = new int[n];

        for (int i = 0; i < n; i++) {

            for (char c : words[i].toCharArray()) {

                masks[i] |= (1 << (c - 'a'));

            }

        }

        int max = 0;

        for (int i = 0; i < n; i++) {

            for (int j = i + 1; j < n; j++) {

                if ((masks[i] & masks[j]) == 0) {

                    max = Math.max(max, words[i].length() *
words[j].length());

                }

            }

        }

        return max;

    }

    public static void main(String[] args) {

        MaxProductWordLengths solution = new MaxProductWordLengths();

        System.out.println(solution.maxProduct(new
String[]{"abcw","baz","foo","bar","xtfn","abcdef"})); // Output: 16

        System.out.println(solution.maxProduct(new
String[]{"a","ab","abc","d","cd","bcd","abcd"})); // Output: 4

        System.out.println(solution.maxProduct(new
String[]{"a","aa","aaa","aaaa"})); // Output: 0

    }

}

```

Question 8

Given a string *s*, reverse only all the vowels in the string and return it. The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.

Example 1:

Input: *s* = "IceCreAm"

Output: "AceCreIm"

Explanation: The vowels in *s* are ['i', 'e', 'e', 'A']. On reversing the vowels, *s* becomes "AceCreIm".

Example 2:

Input: *s* = "leetcode"

Output: "leotcede"

Constraints:

1 <= *s*.length <= 3 * 10⁵

s consist of printable ASCII characters.

```
import java.util.HashSet;
import java.util.Set;

public class ReverseVowels {
    public String reverseVowels(String s) {
        Set<Character> vowels = new HashSet<>();
        vowels.addAll(Set.of('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'));
        char[] chars = s.toCharArray();
        int left = 0, right = chars.length - 1;
        while (left < right) {
            while (left < right && !vowels.contains(chars[left]))
                left++;
            while (left < right && !vowels.contains(chars[right]))
                right--;
            if (left < right) {
                char temp = chars[left];
```

```

        chars[left++] = chars[right];

        chars[right--] = temp;

    }

}

return new String(chars);

}

public static void main(String[] args) {

    ReverseVowels solution = new ReverseVowels();

    System.out.println(solution.reverseVowels("IceCreAm")); //
Output: "AceCreIm"

    System.out.println(solution.reverseVowels("leetcode")); //
Output: "leotcede"

}

}

```

Question 9

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer. You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2`.

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

Input: `s = "3[a]2[bc]"`

Output: `"aaabcbc"`

Example 2:

Input: `s = "3[a2[c]]"`

Output: `"accaccacc"`

Example 3:

Input: `s = "2[abc]3[cd]ef"`

Output: `"abcbcccdcdcdcd"`

Constraints:

1 <= s.length <= 30

s consists of lowercase English letters, digits, and square brackets '['].

s is guaranteed to be a valid input.

All the integers in s are in the range.

```
import java.util.Stack;

public class DecodeString {

    public String decodeString(String s) {

        Stack<Integer> countStack = new Stack<>();
        Stack<String> stringStack = new Stack<>();
        String currentString = "";
        int currentNum = 0;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                currentNum = currentNum * 10 + (c - '0');
            } else if (c == '[') {
                countStack.push(currentNum);
                stringStack.push(currentString);
                currentString = "";
                currentNum = 0;
            } else if (c == ']') {
                StringBuilder temp = new StringBuilder();
                int times = countStack.pop();
                for (int i = 0; i < times; i++) {
                    temp.append(currentString);
                }
                currentString = stringStack.pop() + temp;
            } else {
                currentString += c;
            }
        }
    }
}
```

```

        return currentString;
    }

    public static void main(String[] args) {
        DecodeString solution = new DecodeString();

        System.out.println(solution.decodeString("3[a]2[bc]")); //
Output: "aaabcbc"

        System.out.println(solution.decodeString("3[a2[c]]")); //
Output: "accaccacc"

        System.out.println(solution.decodeString("2[abc]3[cd]ef")); //
Output: "abccabcccdcdcdcd"

    }
}

```

Question 10

Given a string *s* and an integer *k*, return the length of the longest substring of *s* such that the frequency of each character in this substring is greater than or equal to *k*. If no such substring exists, return 0.

Example 1:

Input: *s* = "aaabb", *k* = 3

Output: 3

Explanation: The longest substring is "aaa", as 'a' is repeated 3 times.

Example 2:

Input: *s* = "ababbc", *k* = 2

Output: 5

Explanation: The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.

Constraints:

1 ≤ *s*.length ≤ 104

s consists of only lowercase English letters.

1 ≤ *k* ≤ 105

```

public class LongestSubstringFrequency {

    public int longestSubstring(String s, int k) {

        return helper(s.toCharArray(), 0, s.length(), k);
    }
}

```

```

    }

    private int helper(char[] chars, int start, int end, int k) {

        if (end - start < k) return 0;

        int[] count = new int[26];

        for (int i = start; i < end; i++) {

            count[chars[i] - 'a']++;

        }

        for (int i = start; i < end; i++) {

            if (count[chars[i] - 'a'] < k) {

                int left = helper(chars, start, i, k);

                int right = helper(chars, i + 1, end, k);

                return Math.max(left, right);

            }

        }

        return end - start;

    }

    public static void main(String[] args) {

        LongestSubstringFrequency solution = new
LongestSubstringFrequency();

        System.out.println(solution.longestSubstring("aaabb", 3)); //
Output: 3

        System.out.println(solution.longestSubstring("ababbc", 2)); //
Output: 5

    }

}

```